

YES — these are exactly the right questions. 😊 You're now asking "**what is actually provided by the library vs what am I supposed to create?**" That's a very important distinction.

Let's separate everything.

1. Are `@auth/prisma-adapter` and `@prisma/client` default?

No. They are not both defaults.

This is where my previous example could have been clearer.

`@auth/prisma-adapter`

This is a **separate npm package** provided for integrating Auth.js with Prisma.

You would explicitly install it.

Conceptually:

```
npm install next-auth @auth/prisma-adapter @prisma/client
```

You don't magically get:

```
import { PrismaAdapter } from "@auth/prisma-adapter";
```

```
const adapter = new PrismaAdapter(db);
```



just because you installed `next-auth`.

`@/lib/prisma`

This one is **your own application file**.

It isn't a package.

For example, **you might create**:

```
my-app/
├── app/
├── auth.ts
├── lib/
│   └── prisma.ts ← YOU create this
└── package.json
```



Then:

```
// lib/prisma.ts

import { PrismaClient } from "@prisma/client";

export const prisma = new PrismaClient();
```

And now this:

```
import { prisma } from "@/lib/prisma";
```

means:

"Import the Prisma client that **I created in my project**."

The `@/` part is normally a TypeScript/Next.js path alias pointing to your project root (depending on your `tsconfig.json` / `jsconfig.json`).

2. So how do I know what is provided by Auth.js?

This is a **really good developer habit**:

Don't guess based on examples.

Check:

Official documentation

For example, Auth.js documentation tells you:

```
import NextAuth from "next-auth";
```

and:

```
import Google from "next-auth/providers/google";
```

Those are provided by the `next-auth` package.

Then if documentation says:

```
import { PrismaAdapter } from "@auth/prisma-adapter";
```

you know that's a **separate package**.

You can also check `package.json`.

For example:

```
{
  "dependencies": {
    "next-auth": "...",
    "@auth/prisma-adapter": "...",
    "@prisma/client": "..."
  }
}
```

That tells you which packages your project actually installed.

And your own imports:

</> TypeScript



point to files in **your own codebase**.

3. Now your bigger question:

Why do we need an adapter? Why can't Auth.js simply send something to the Next.js server, and Next.js sends it to the database?

This is the key misunderstanding.

Auth.js IS already running inside your Next.js server.

There isn't necessarily:

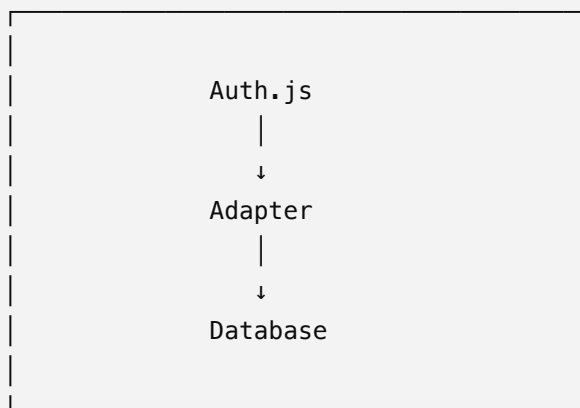
```
graph TD; A[Auth.js] --> B[Next.js Server]; B --> C[Database];
```



as three separate servers.

Instead:

NEXT.JS SERVER



Auth.js is **server-side code** running as part of your Next.js application.

4. Then why does Auth.js need an adapter?

Because Auth.js shouldn't have to know:

"Which database technology does YOUR application use?"

Imagine your application uses:

PostgreSQL + Prisma



Another company uses:

MySQL + Drizzle



Another:

MongoDB



Another:

DynamoDB



Auth.js wants to provide the authentication machinery without hardcoding one database implementation.

So it defines an interface roughly like:

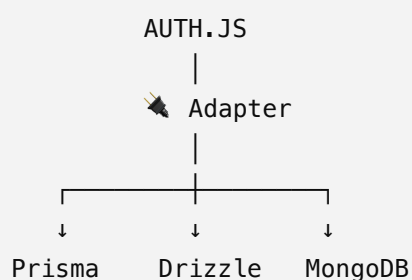
```
Auth.js
  |
  | "I need to create a user."
  ↓
Adapter
  |
  | translates that operation
  ↓
Your database
```

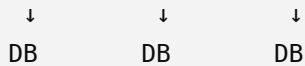


5. Think of the adapter as a plug

This analogy is actually useful. 🚧

Auth.js is:





Auth.js doesn't need to know all the details of every database.

It just knows:

```
createUser()  
getUser()  
createSession()  
getSession()  
deleteSession()  
...
```



The adapter implements those operations for a particular database setup.

6. Why couldn't Auth.js just use Prisma directly?

It **could**, if Auth.js decided:

"Everyone must use Prisma."

But that's a terrible restriction for a general authentication library.

Auth.js would then effectively be:

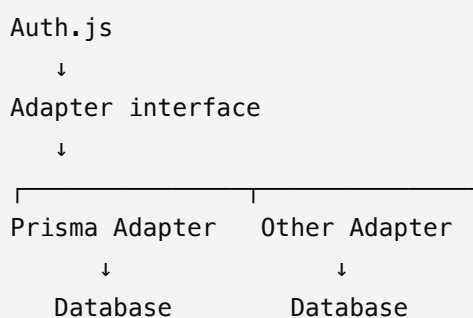
```
Auth.js  
  ↓  
Prisma  
  ↓  
PostgreSQL/MySQL/etc.
```



What if your application doesn't use Prisma?

You'd be stuck.

Instead:



That's why adapters exist.

7. And now the API routes question 🔥

You asked:

```
/api/auth/signin  
/api/auth/callback/google  
/api/auth/signout
```

Where did these come from? We didn't write them.

Excellent observation.

You DID write the route handler:

```
// app/api/auth/[...nextauth]/route.ts  
  
import { handlers } from "@auth";  
  
export const { GET, POST } = handlers;
```

But you didn't individually write:

```
/api/auth/signin  
/api/auth/signout  
/api/auth/callback/google
```



because Auth.js's `handlers` implement the authentication routes for you.

8. What does `[...nextauth]` mean?

This is a Next.js **catch-all dynamic route**.

You have:

```
app/  
└─ api/  
    └─ auth/  
        └─ [...nextauth]/  
            └─ route.ts
```



The:

```
[...nextauth]
```



means:

"Match multiple path segments underneath `/api/auth.`"

So requests such as:

```
/api/auth/signin  
/api/auth/signout  
/api/auth/callback/google
```



can reach this Route Handler.

9. Then Auth.js decides what to do

You wrote:

```
export const { GET, POST } = handlers;
```

`handlers` comes from:

```
const { handlers } = NextAuth(...)
```

Auth.js provides those handlers.

So conceptually:

```
Browser  
|  
| GET /api/auth/signin  
↓  
Next.js routing  
|  
↓  
[...nextauth]/route.ts  
|  
↓  
handlers.GET  
|
```



```
↓  
Auth.js  
|  
↓  
"Okay, this is a signin request"
```

For:

```
/api/auth/callback/google
```



Auth.js recognizes:

```
callback  
provider = google
```



and runs its callback logic.

10. So are those APIs "default"?

They are provided by Auth.js when you expose its handlers through your Route Handler.

They aren't generic Next.js APIs.

Next.js itself does NOT automatically give you:

```
/api/auth/signin  
/api/auth/signout  
/api/auth/callback/google
```



If you installed plain Next.js without Auth.js, those routes don't exist.

It's the combination:

```
Auth.js  
+  
your Route Handler exposing Auth.js handlers
```



that makes them available.

11. Let's make this concrete

You write:

```
// auth.ts

import NextAuth from "next-auth";
import Google from "next-auth/providers/google";

export const { handlers, auth, signIn, signOut } = NextAuth({
  providers: [
    Google({
      clientId: process.env.AUTH_GOOGLE_ID!,
      clientSecret: process.env.AUTH_GOOGLE_SECRET!,
    }),
  ],
});
```

Auth.js gives you:

```
handlers
auth
signIn
signOut
```



Then you write:

```
// app/api/auth/[...nextauth]/route.ts

import { handlers } from "@auth";

export const { GET, POST } = handlers;
```

Now you've effectively connected:

```
/api/auth/*
  ↓
Auth.js handlers
```



And Auth.js knows how to handle its supported auth operations.

12. What if we DIDN'T write the Route Handler?

Suppose you have:

```
auth.ts
```



but DON'T have:

```
app/api/auth/[...nextauth]/route.ts
```



Then those HTTP endpoints aren't exposed through that route.

Your browser can't magically access:

```
/api/auth/signin
```



just because Auth.js is installed.

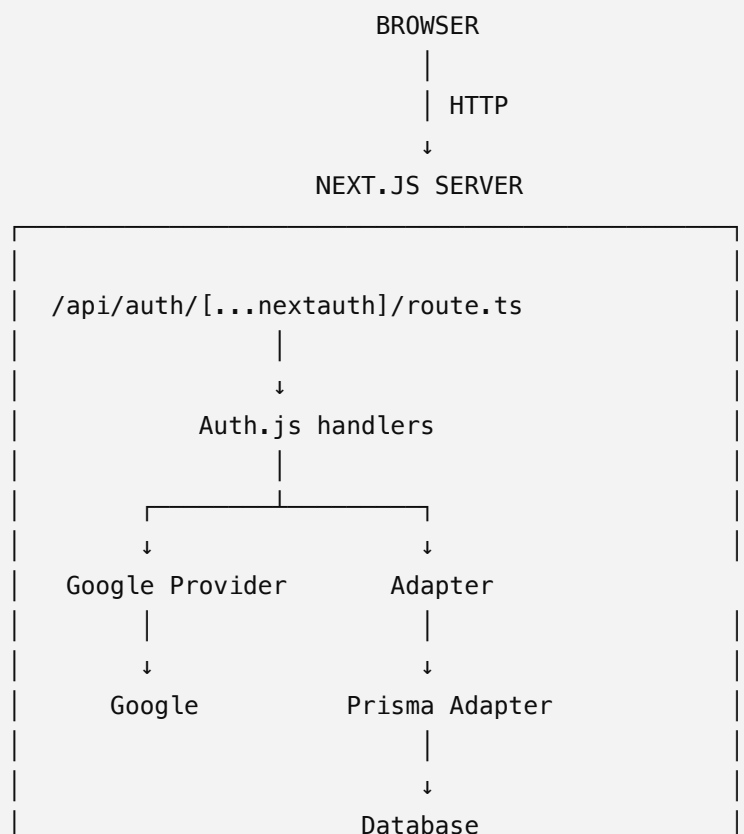
You have to wire Auth.js into Next.js.

That's exactly what this does:

```
export const { GET, POST } = handlers;
```

13. The entire architecture now

This should look much clearer:



14. One more subtle point: not every Auth.js setup needs an adapter

This is important.

If you use a session strategy that doesn't require database-backed sessions, you may not configure a database adapter at all.

For example, conceptually:

```
NextAuth({  
  providers: [Google(...)],  
  session: {  
    strategy: "jwt",  
  },  
});
```

Here, Auth.js can maintain session information using its JWT-based strategy rather than storing session records in your database.

You might still have a database for application users depending on your setup, but **an adapter is not universally required just to use Auth.js.**

So don't memorize:

❌ "Auth.js always requires an adapter."

Instead:

✅ "An adapter is used when Auth.js needs to persist authentication data in a database."

15. Your exact questions answered

Q: Is `@auth/prisma-adapter` default?

No.

It's a separate package you install when using the Prisma adapter.

Q: Is `@lib/prisma` default?

No.

That's typically a file **you create** to initialize/export your Prisma client.

Q: Why adapter?

Because Auth.js shouldn't be tied to one database/ORM.

```
Auth.js
  ↓
Adapter
  ↓
Your DB technology
```



Q: Why doesn't Auth.js just go through Next.js server?

Because Auth.js is already server-side code running inside your Next.js application.

It's more like:

```
Next.js server
├── Auth.js
│   └── Adapter
│       └── DB
└── Your other server code
```



not three separate servers.

Q: Where did `/api/auth/signin` etc. come from?

Auth.js provides the handlers.

You expose them through:

```
export const { GET, POST } = handlers;
```

inside:

```
app/api/auth/[...nextauth]/route.ts
```



Q: Are they built into Next.js?

No.

They're **Auth.js** routes exposed through a **Next.js** Route Handler.

The most important mental model

There are **three different things** here:

1. **NEXT.JS**
 - └ Provides the HTTP routing mechanism
2. **AUTH.JS**
 - └ Provides authentication/session logic
3. **ADAPTER**
 - └ Provides the database persistence implementation



So when you see:

```
export const { GET, POST } = handlers;
```

think:

"I'm telling Next.js: for this URL, use Auth.js's already-implemented HTTP handlers."

And when you see:

```
adapter: PrismaAdapter(prisma)
```

think:

"I'm telling Auth.js: when you need to save/retrieve auth data, use this database bridge."

That's the clean separation. 🔥



```
// app/api/auth/[...nextauth]/route.ts

import { handlers } from "@auth";

export const { GET, POST } = handlers; just by writing this
how api/auth/signin singout callback works i really cant
understnad..and whats ORM??
```

YES — the missing piece is **how one GET / POST handler can magically handle many different URLs**. It isn't magic, but the combination of **Next.js catch-all routing + Auth.js's handler** hides a lot of code from you.

Let's slow this way down. 😊

1. First, forget Auth.js

Imagine you wrote this yourself:

```
</> TypeScript

// app/api/auth/[...nextauth]/route.ts

export async function GET(request: Request) {
  return Response.json({
    message: "Hello"
  });
}
```

And your folder is:

```
app/
├── api/
│   └── auth/
│       ├── [...nextauth]/
│       └── route.ts
```

What does:

```
[...nextauth]
```

mean?

It's a **catch-all route**.

It tells Next.js:

"Whatever path comes after `/api/auth/`, send the request to this `route.ts`."

So:

```
/api/auth/signin
```



matches.

```
/api/auth/signout
```



matches.

```
/api/auth/callback/google
```



matches.

```
/api/auth/anything/else
```



also matches.

2. Think of it like a receptionist 🏢

Imagine this URL:

```
/api/auth/*
```



The `*` basically means:

"Anything after here."

So:

```
/api/auth/signin  
/api/auth/signout  
/api/auth/callback/google
```



all enter the same building:

```
app/api/auth/[...nextauth]/route.ts
```



Next.js is responsible for **routing the request there**.

3. But then how does `signin` behave differently from `signout`?

This is the part you were missing.

Suppose we wrote our own handler:

`</>` TypeScript 

```
export async function GET(request: Request) {  
  const url = new URL(request.url);  
  
  console.log(url.pathname);  
  
  return Response.json({  
    path: url.pathname  
  });  
}
```

If the browser requests:

GET /api/auth/signin 

the handler receives:

```
url.pathname  
=  
"/api/auth/signin" 
```

If the browser requests:

GET /api/auth/callback/google 

the same handler receives:

```
url.pathname  
=  
"/api/auth/callback/google" 
```

So the handler can inspect **which path was requested**.

4. Imagine WE wrote Auth.js ourselves

It could conceptually have code like:

</> TypeScript



```
export async function GET(request: Request) {  
  
  const url = new URL(request.url);  
  
  if (url.pathname === "/api/auth/signin") {  
    return handleSignIn(request);  
  }  
  
  if (url.pathname === "/api/auth/callback/google") {  
    return handleGoogleCallback(request);  
  }  
  
  if (url.pathname === "/api/auth/session") {  
    return handleSession(request);  
  }  
  
  // ...  
}
```

And maybe:

</> TypeScript



```
async function handleSignIn(request) {  
  // create OAuth authorization URL  
  // redirect to Google  
}
```

</> TypeScript



```
async function handleGoogleCallback(request) {  
  // receive authorization code  
  // exchange code with Google  
  // find/create user  
  // create session  
  // set cookie  
}
```

</> TypeScript



```
async function handleSession(request) {  
  // read session cookie  
  // find session
```

```
// return session  
}
```

That is basically the conceptual idea of what Auth.js's handlers are doing for you.

You don't see all that implementation because Auth.js already wrote it.

5. Now look at this line again

```
</> TypeScript  
  
import { handlers } from "@auth";
```

Where did `handlers` come from?

From your:

```
</> TypeScript  
  
// auth.ts  
  
import NextAuth from "next-auth";  
  
export const {  
  handlers,  
  auth,  
  signIn,  
  signOut  
} = NextAuth({  
  providers: [  
    Google(...)  
  ]  
});
```

When you call:

```
</> TypeScript  
  
NextAuth({...})
```

Auth.js creates a bunch of functionality for your application.

One of the things it gives you is:

```
handlers
```

Conceptually:

```
handlers
├── GET
└── POST
```



Those functions contain Auth.js's authentication routing logic.

6. Then this line:

</> TypeScript

```
export const { GET, POST } = handlers;
```



means:

"Next.js, for this Route Handler, use Auth.js's GET function and Auth.js's POST function."

That's it.

You're not creating the authentication logic here.

You're **connecting** Auth.js's existing logic to Next.js's routing system.

7. Think about it like plugging a machine into a wall

Auth.js gives you:

AUTH.JS

```
GET function
POST function
```

handlers



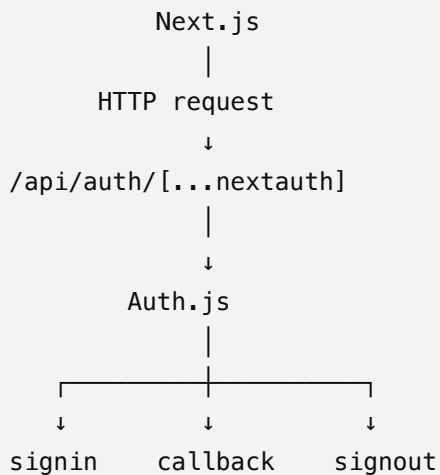
Your Route Handler says:

</> TypeScript

```
export const { GET, POST } = handlers;
```



Now:



8. Let's trace `/api/auth/signin`

Browser sends:

```
</> http  
GET /api/auth/signin
```



Step 1

Next.js sees:

```
app/api/auth/[...nextauth]/route.ts
```



and says:

```
"This URL matches that catch-all route."
```

Step 2

The request is sent to the exported:

```
</> TypeScript  
GET
```



Step 3

But your `GET` is actually:

```
</> TypeScript
```



`handlers.GET`

from Auth.js.

Step 4

Auth.js looks at the requested path/action.

Conceptually:

```
/api/auth/signin
  ↓
Auth.js
  ↓
"Ah, sign-in request."
```



Step 5

Auth.js executes its sign-in logic.

For Google:

```
Auth.js
  ↓
construct Google authorization URL
  ↓
redirect browser
  ↓
Google
```



9. Now `/api/auth/callback/google`

Browser comes back from Google:

```
</> http
GET /api/auth/callback/google?code=ABC123
```



Next.js:

```
/api/auth/callback/google
  ↓
[...nextauth]
  ↓
route.ts
```



↓
GET

Again:

`</> TypeScript`

`GET === handlers.GET`

Then Auth.js sees:

`callback/google`

and knows:

"This is the Google callback."

Then internally:

```

authorization code
  ↓
exchange with Google
  ↓
get identity
  ↓
find/create user
  ↓
create session
  ↓
set cookie
  ↓
redirect user
  
```

10. Now `/api/auth/signout`

Suppose:

`POST /api/auth/signout`

Next.js again:

```

/api/auth/signout
  ↓
[...nextauth]
  ↓
POST handler
  
```

↓
Auth.js

Auth.js recognizes:

signout



and performs its sign-out logic.

Conceptually:

```
session
  ↓
invalidate/delete session
  ↓
clear cookie
  ↓
redirect
```



11. So the key is NOT this line alone

You were looking at:

```
</> TypeScript
export const { GET, POST } = handlers;
```



and thinking:

"How can these two lines create signin, signout and callback APIs?!"

They **don't** create the logic.

They connect the logic.

The actual logic was created inside:

```
</> TypeScript
NextAuth({...})
```



by the Auth.js package.

Think:

```

NextAuth()
  ↓
creates authentication machinery
  ↓
returns handlers
  ↓
you export those handlers
  ↓
Next.js can invoke them

```



12. And `[...nextauth]` is the other half

You need BOTH pieces:

Catch-all route:

```
[...nextauth]
```



says:

"Send `/api/auth/*` requests here."

Auth.js handlers:

```

</> TypeScript

export const { GET, POST } = handlers;

```



says:

"When they arrive here, let Auth.js process them."

Together:

```

/api/auth/signin
  |
  ↓
[...nextauth]
  |
  ↓
Auth.js GET handler
  |
  ↓
Sign-in logic

```



and:

```

/api/auth/callback/google
  |
  ↓
[...nextauth]
  |
  ↓
Auth.js GET handler
  |
  ↓
Callback logic

```



13. A real-world analogy

Imagine a hospital. 🏥

You have one reception desk:

```
/api/auth/[...nextauth]
```



Everyone enters through that desk.

```

Patient: "I need sign-in."
Patient: "I need sign-out."
Patient: "I'm returning from Google."

```



The receptionist:

```
Auth.js handler
```



looks at the request and routes it to the appropriate internal procedure.

You don't build a separate door for:

```

/api/auth/signin
/api/auth/signout
/api/auth/callback/google

```



The catch-all route is the **one entrance**.

Auth.js is the **receptionist who knows what to do with each request**.

14. Now: what's an ORM?

This is another important concept.

ORM = Object-Relational Mapping.

It's a tool that lets your application interact with a relational database using programming-language objects/methods instead of writing raw SQL for every operation.

For example, without an ORM:

```
</> SQL

SELECT *
FROM users
WHERE email = 'gopi@example.com';
```



Your JavaScript/TypeScript application has to deal with SQL directly.

With an ORM such as **Prisma**, you can write:

```
</> TypeScript

const user = await prisma.user.findUnique({
  where: {
    email: "gopi@example.com",
  },
});
```



Prisma turns that into the appropriate database query.

Conceptually:

```
Your TypeScript
  ↓
  Prisma
  ↓
  SQL
  ↓
  PostgreSQL
```



15. So Prisma is an ORM

For example:

</> TypeScript



```
const user = await prisma.user.findUnique({
  where: {
    email: "gopi@example.com",
  },
});
```

You are expressing:

"Find the user whose email is this."

Prisma handles translating that into database operations.

16. Why do we need an ORM?

You don't **have** to use one.

You could write:

</> TypeScript



```
await db.query(
  "SELECT * FROM users WHERE email = $1",
  ["gopi@example.com"]
);
```

That's perfectly possible.

But ORMs provide things such as:

- typed database queries
- schema modeling
- relationships
- migrations/tooling
- less repetitive SQL
- easier application-level database access

So:

Prisma
Drizzle
TypeORM
Sequelize



are examples of database tools/ORMs (with some differences in how they work).

17. And now connect ORM + Adapter

This is where all the pieces finally line up. 🔥

Suppose:

```
PostgreSQL
  ↑
Prisma
  ↑
Prisma Adapter
  ↑
Auth.js
  ↑
Next.js
```



More accurately:

```
NEXT.JS SERVER
  |
  ↓
Auth.js
  |
  ↓
Prisma Adapter
  |
  ↓
Prisma
  |
  ↓
PostgreSQL
```



Each layer has a different job.

Auth.js

Authentication/session logic



Adapter

Translate Auth.js persistence operations
into the database adapter's API



Prisma

Interact with your database



PostgreSQL

Actually store the data



18. Example: creating a session

Conceptually:

```
Auth.js
  |
  | "Create session for user 123"
  ↓
Prisma Adapter
  |
  | translates operation
  ↓
Prisma
  |
  | database query
  ↓
PostgreSQL
```



Database:

```
sessions
-----
sessionToken | userId | expires
ABC123       | 123    | ...
```



Then Auth.js can send the corresponding session cookie to the browser.



The whole thing in one picture

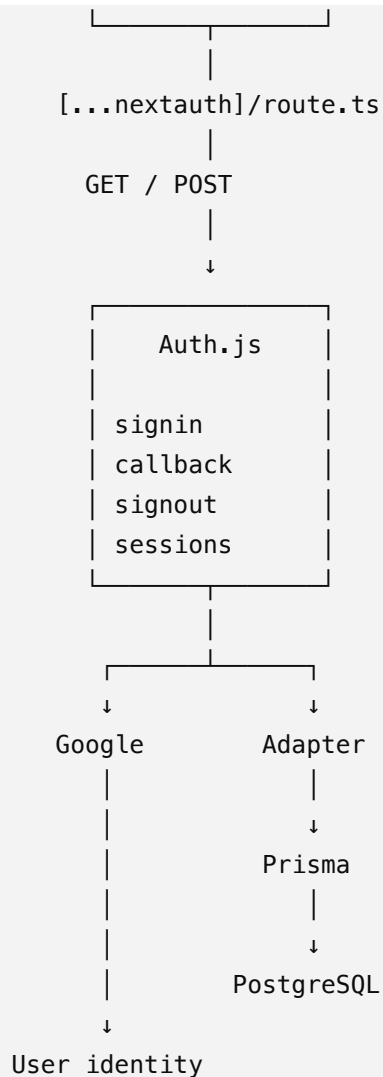
BROWSER

|
| HTTP
↓

Next.js

/api/auth/*





The key realization:

`route.ts` doesn't contain the authentication implementation.

It says:

"Next.js, when requests arrive at `/api/auth/*`, hand GET/POST requests to `Auth.js`."

And:

`[...nextauth]` doesn't create those APIs either.

It tells Next.js:

"All paths underneath `/api/auth/` should come to this handler."

Then `Auth.js`'s already-written code decides whether the request means signin, callback, signout, session, etc.

That's why those APIs appear even though you didn't manually write 100 different route files. 😊



Awesome 😄 Let's move to the next important Auth.js concept: how the session is actually used to protect pages and APIs.

Lesson 7.x — Protecting Routes with Auth.js

We've now understood:

```
Google
  ↓
OAuth/OIDC
  ↓
Auth.js callback
  ↓
User/Account
  ↓
Session
  ↓
Cookie
```



Now the natural question is:

Once the user is logged in, how do I actually stop an unauthenticated user from accessing `/dashboard` or `/api/orders` ?

This is where `auth()` becomes really important.

1. Protecting a Server Component

Suppose we have:

```
app/
├── login/
│   └── page.tsx
└── dashboard/
    └── page.tsx
```



Our dashboard should only be accessible to logged-in users.

We can write:

```
<> TypeScript

// app/dashboard/page.tsx

import { auth } from "@auth";
import { redirect } from "next/navigation";
```



```
export default async function DashboardPage() {
  const session = await auth();

  if (!session) {
    redirect("/login");
  }

  return (
    <div>
      <h1>Welcome {session.user?.name}</h1>
      <p>{session.user?.email}</p>
    </div>
  );
}
```

Now let's understand **exactly what happens**.

2. User requests `/dashboard`

Browser:

```
</> http

GET /dashboard
Cookie: authjs.session-token=ABC123
```



Next.js receives this request.

Because `dashboard/page.tsx` is a **Server Component**, it executes on the server.

Then:

```
</> TypeScript

const session = await auth();
```



runs.

3. What does `auth()` do?

Remember where it came from:

```
</> TypeScript

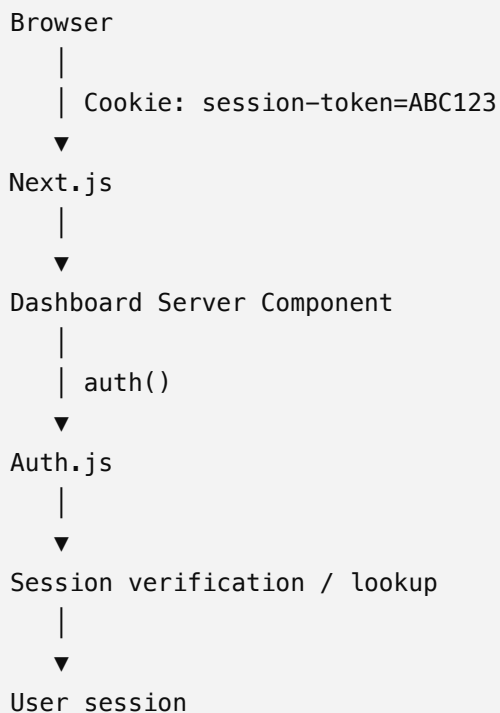
// auth.ts
```




```
export const {
  handlers,
  auth,
  signIn,
  signOut
} = NextAuth({...});
```

So `auth()` is an Auth.js server-side function.

Conceptually:



If the session is valid:

```
</> TypeScript

session = {
  user: {
    name: "Gopi",
    email: "gopi@example.com"
  }
}
```

Then the page renders.

4. What if the user isn't logged in?

Suppose there is no valid session.

Then:

```
</> TypeScript
```



```
const session = await auth();
```

returns something equivalent to:

```
</> TypeScript
```



```
null
```

So:

```
</> TypeScript
```



```
if (!session) {  
  redirect("/login");  
}
```

executes.

Next.js sends a redirect response.

Conceptually:

```
</> http
```



```
307 Temporary Redirect  
Location: /login
```

The browser then requests:

```
</> http
```



```
GET /login
```

So:

```
User → /dashboard
```



```
  ↓  
auth()  
  ↓  
no session  
  ↓  
redirect("/login")  
  ↓  
/login
```

5. Why isn't this authorization?

This is an important distinction.

This code checks:

```
</> TypeScript
```



```
if (!session)
```

That is **authentication**.

We're asking:

"Is this person logged in?"

But suppose we have:

Admin Dashboard



and only admins can access it.

Then we need **authorization**.

For example:

```
</> TypeScript
```



```
const session = await auth();
```

```
if (!session) {  
  redirect("/login");  
}
```

```
if (session.user.role !== "ADMIN") {  
  redirect("/forbidden");  
}
```

Now we're asking two different questions:

Authentication:

"Who are you / are you logged in?"



Authorization:

"Are you allowed to do this?"

6. But there's an important security rule

Never rely only on hiding UI.

For example:

```
</> TypeScript

{session?.user.role === "ADMIN" && (
  <button>Delete User</button>
)}
```



This is useful for UX.

But it is **not security**.

A malicious user can manually send a request to your server.

For example:

```
</> http

POST /api/users/delete
```



Therefore your server-side action/API must check authorization too.

7. Protecting a Route Handler

Suppose we have:

```
app/api/orders/route.ts
```



We can do:

```
</> TypeScript

import { auth } from "@auth";
import { NextResponse } from "next/server";

export async function POST(request: Request) {
  const session = await auth();

  if (!session) {
    return NextResponse.json(
      { error: "Unauthorized" },
      { status: 401 }
    );
  }
}
```

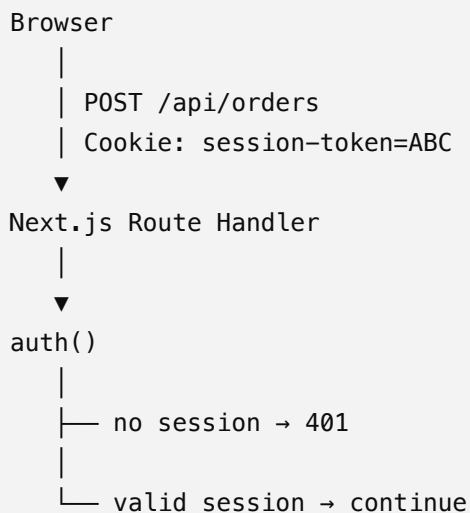


```
}

// User is authenticated
// Continue processing order...

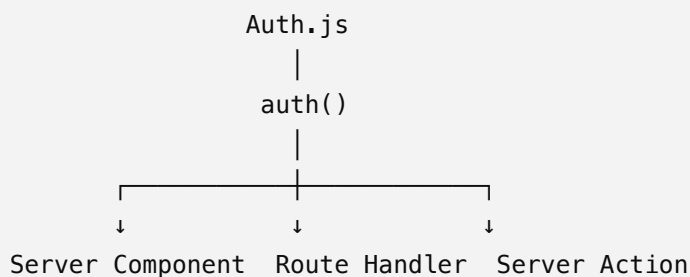
return NextResponse.json({
  success: true,
});
}
```

Now:



8. This is why `auth()` is so useful

You can use the same server-side authentication context in multiple places:



For example:

Server Component

```
</> TypeScript

const session = await auth();
```

Route Handler

</> TypeScript



```
const session = await auth();
```

Server Action

</> TypeScript



```
const session = await auth();
```

The exact mechanics differ depending on the execution context, but the idea is:

Your server-side code asks Auth.js to resolve the current authenticated user/session.

9. Server Action example

Suppose:

</> TypeScript



```
"use server";

import { auth } from "@auth";

export async function deleteAccount() {
  const session = await auth();

  if (!session) {
    throw new Error("Unauthorized");
  }

  // Delete account...
}
```

Even if your UI hides the "Delete Account" button from logged-out users, the Server Action still checks authentication.

That's the security boundary.

10. Interview answer 🎯

If an interviewer asks:

"How would you protect a page in Next.js using Auth.js?"

A strong answer would be:

"For server-rendered pages, I can call Auth.js's server-side `auth()` function to resolve the current session. If there's no valid session, I redirect the user to the login page. For protected mutations or APIs, I perform the authentication and authorization checks again inside the Server Action or Route Handler rather than relying on client-side UI checks."

And if they ask:

"Is hiding an admin button authorization?"

Say:

"No. That's only a UI concern. Authorization must be enforced on the server at the resource or mutation boundary."

One thing I want you to notice

We now have **three different places** where Auth.js can participate:

1. LOGIN

```
signIn()  
  ↓  
OAuth  
  ↓  
callback  
  ↓  
session created
```

2. READING SESSION

```
auth()  
  ↓  
"Who is currently logged in?"
```

3. PROTECTING ACTIONS

```
auth()  
  ↓  
authenticate  
  ↓  
authorize
```



↓
perform operation

That distinction is **very interview-important**.

Next after this

The natural next piece is **Auth.js callbacks** — specifically:

</> TypeScript



```
callbacks: {  
  async jwt() {},  
  async session() {},  
  async signIn() {},  
  async redirect() {}  
}
```

Because you've now seen `session` being created and consumed, the next question is:

"How do I customize what gets stored in the JWT/session — for example, how do I put a user's `role` or `id` into the session?"

That's where the `jwt()` and `session()` callbacks finally make sense. 🔥



how many more are there

Not too many 😊 We're actually pretty close to wrapping the **Authentication & Authorization** section.

Based on what we've covered so far, I'd structure the remaining part like this:

🔑 Remaining Auth.js / Auth & Authorization lessons

#	Topic	What you'll learn
7.6	🛡️ Route Protection	<code>auth()</code> , protecting pages, APIs, Server Actions
7.7	🔄 Auth.js Callbacks	<code>signIn</code> , <code>jwt</code> , <code>session</code> , <code>redirect</code> callbacks
7.8	👤 Session Customization	Put <code>user.id</code> , <code>role</code> , etc. into the session

#	Topic	What you'll learn
7.9	 Authorization / RBAC	Roles, permissions, admin/user access
7.10	 Session Strategies	Database sessions vs JWT sessions
7.11	 CSRF + SameSite	Why state-changing requests need protection
7.12	 Middleware / Proxy Auth	Protecting groups of routes before rendering
7.13	 Logout & Session Expiry	Sign-out, expiration, invalid/expired sessions
7.14	 Complete Auth Architecture	Put Browser → Next.js → Auth.js → OAuth → DB → Session together
7.15	 Auth Interview Questions	Real interview scenarios + architecture questions

So **about 10 lessons remain**, depending on how deeply we split things.

But some of these are short. The **big ones** are:

1. Callbacks
2. Session customization
3. Authorization/RBAC
4. JWT vs DB sessions
5. CSRF/SameSite
6. Middleware/Proxy

After that, I'd consider the Auth section **complete** and move you to the next major Next.js/interview area. 🚀

And I wouldn't rush through these—especially callbacks and authorization. Given the way you've been learning, I'd rather make sure you can explain the **actual request flow** than just memorize Auth.js APIs.



so after this authentication and authorization are done?

Yes. 👍 After we finish those remaining Auth.js topics, we can consider the **Authentication & Authorization** section complete.

More precisely, your **Next.js Auth & Authorization roadmap** will cover:

AUTHENTICATION



- ✓ Authentication vs Authorization
- ✓ Auth architecture
- ✓ Cookies
- ✓ Sessions
- ✓ JWT
- ✓ Password hashing
- ✓ Login flow
- ✓ Server Components + auth
- ✓ Client Components + auth
- ✓ Auth.js
- ✓ OAuth 2.0
- ✓ OpenID Connect
- ✓ Google authentication
- ✓ Authorization code + callback
- ✓ Client ID / Client Secret
- ✓ Token endpoint
- ✓ Auth.js adapters
- ✓ ORM + database persistence
- ✓ User / Account / Session
- ✓ Database sessions vs JWT sessions
- ✓ Route protection
- ✓ Auth.js callbacks
- ✓ Session customization

AUTHORIZATION

- ✓ Authentication vs authorization
- ✓ Protecting Server Components
- ✓ Protecting Route Handlers
- ✓ Protecting Server Actions
- ✓ Roles / RBAC
- ✓ Permissions
- ✓ Server-side authorization
- ✓ Client UI vs actual security boundary

SECURITY

- ✓ HttpOnly / Secure / SameSite
- ✓ CSRF
- ✓ Session expiry

- ✓ Logout
- ✓ Middleware / Proxy protection
- ✓ Common auth vulnerabilities

Then we'll do **one complete end-to-end architecture walkthrough**:

```
Browser
  ↓
Login
  ↓
Auth.js
  ↓
Google OAuth/OIDC
  ↓
Callback
  ↓
Google token exchange
  ↓
User + Account
  ↓
Adapter
  ↓
Database
  ↓
Session
  ↓
Cookie
  ↓
Protected request
  ↓
auth()
  ↓
Authentication
  ↓
Authorization
  ↓
Protected resource
```



That final walkthrough is important because you've been asking the right kind of **"but who actually calls what?"** questions. 😊 It will tie all the individual pieces together.

So yes: once we finish these, you can confidently mark *Next.js Authentication & Authorization* as DONE. 🗝️✅



ChatGPT can make mistakes. Check important info.

